

30-Day Python Teaching Plan

PCEP — Python Certified Entry Professional

Target: University First-Year Student | Session: 30–45 min/day (Online, Live Coding)

Prepared for Trainer Use — Read the night before each session

Phase	Days	Focus
Phase 1 — Foundation	1–7	Python basics, variables, loops, conditionals
Phase 2 — Core Python	8–18	Lists, dicts, functions, errors, modules, files
Phase 3 — Intermediate	19–25	OOP, sets, recursion, iterators, regex
Phase 4 — Exam Prep	26–30	PCEP deep dive, two mock exams, final revision

PHASE 1 — FOUNDATION (DAYS 1–7)

Day 1

Foundation

Welcome to Python & Setup

LEARNING GOAL

Student understands what Python is, why it matters for the PCEP exam, and can write and run their first program on their own machine.

TOPICS TO COVER

- What is Python? Interpreted, high-level, readable
- Installing Python 3 & VS Code (or IDLE)
- The Python interactive shell vs script files
- `print()` — your first function
- Comments with `#`
- How Python reads code top to bottom

CODE EXAMPLE

```
# My first Python program
print("Hello, World!")
print("My name is Alex")
print(2 + 3) # Python can do math too
```

IN-SESSION SCENARIO

Ask the student: 'What do you think `print()` does?' Let them guess before running it. Then change the text inside quotes — show that Python obeys exactly what you tell it.

HOMEWORK

Write a program that prints their name, age, and favourite hobby on three separate lines.

PCEP SYLLABUS COVERAGE

Section 1: Computer programming & Python basics

Trainer tip: Spend 10 min just on the shell. Type `2+2`, type `'hello'`, let them play. Curiosity first, syntax later.

Day 2

Foundation

Variables & Data Types

LEARNING GOAL

Student can create variables, understand the 4 basic data types, and use `type()` to inspect values.

TOPICS TO COVER

- Variables: naming rules, assignment with `=`
- `int` — whole numbers
- `float` — decimal numbers
- `str` — text in quotes
- `bool` — `True` / `False`
- `type()` function
- Variable reassignment

CODE EXAMPLE

```
name = "Alex"
age = 19
gpa = 3.75
is_student = True

print(name)
print(type(age)) # <class 'int'>
print(type(gpa)) # <class 'float'>
print(type(name)) # <class 'str'>
```

IN-SESSION SCENARIO

Real world: 'Think of variables like labelled boxes. You put something in a box, give it a label, and can always find it by that label.' Ask: what box would you need for a student registration form?

HOMEWORK

Create variables for: a product name, its price, quantity in stock, and whether it is on sale. Print each with its type.

PCEP SYLLABUS COVERAGE

Section 2: Data types, variables, basic I/O

Trainer tip: Watch for confusion between = (assignment) vs == (comparison). Plant the seed today, they will need it for Day 8.

Day 3

Foundation

User Input & String Basics

LEARNING GOAL

Student can take input from the user, convert types, and do basic string operations.

TOPICS TO COVER

- `input()` always returns a string
- `int()` and `float()` for type conversion
- String concatenation with `+`
- f-strings for clean output
- `len()` to count characters
- String indexing: `name[0]`

CODE EXAMPLE

```
name = input("What is your name? ")
age = int(input("How old are you? "))

print(f"Hello {name}, you are {age}
years old.")

print(f"Next year you will be {age +
1}.")

print(f"Your name has {len(name)}
letters.")
```

IN-SESSION SCENARIO

Build a mini 'profile card' together: ask for name, age, city. Print a formatted card. The student sees immediate real-world output.

HOMEWORK

Build a simple bill calculator: ask for item price and quantity, print the total with a formatted message.

PCEP SYLLABUS COVERAGE

Section 2: Basic I/O, type casting

Trainer tip: Demo the error when you skip `int()` — show the ugly `TypeError`. Let them see WHY conversion is needed.

Day 4

Foundation

Operators

LEARNING GOAL

Student can use all arithmetic, comparison, and logical operators correctly.

TOPICS TO COVER

- Arithmetic: + - * / // % **
- // floor division vs / true division
- % modulus (remainder)
- ** exponentiation
- Comparison: == != > < >= <=
- Logical: and or not
- Operator precedence (BODMAS applies)

CODE EXAMPLE

```
a, b = 17, 5

print(a + b) # 22
print(a / b) # 3.4
print(a // b) # 3 (floor)
print(a % b) # 2 (remainder)
print(a ** 2) # 289

print(a > 10 and b < 10) # True
print(a == 17 or b == 99) # True
print(not (a == b)) # True
```

IN-SESSION SCENARIO

Even/odd checker: 'A number is even if dividing by 2 leaves 0 remainder. How do we check that?' Guide them to discover `number % 2 == 0`.

HOMEWORK

Write a program that takes a number and prints: is it even or odd, is it greater than 100, and what is its square.

PCEP SYLLABUS COVERAGE

Section 2: Operators and expressions

Trainer tip: % (modulus) confuses beginners. Use a clock analogy — 14 o'clock mod 12 = 2, just like 14 % 12 = 2.

Day 5

Foundation

if / elif / else

LEARNING GOAL

Student can write conditional logic with proper indentation and trace through branching code.

TOPICS TO COVER

- if statement structure
- else clause
- elif for multiple branches
- Indentation is mandatory in Python (4 spaces)
- Nested if statements
- Truthy and falsy values

CODE EXAMPLE

```
score = int(input("Enter your score: "))

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 50:
    print("Grade: D")
else:
    print("Grade: F")
print("Please revise and retry.")
```

IN-SESSION SCENARIO

Grade calculator — student enters a score and gets their grade. Then extend it: 'What if we also want to print a message for distinction (≥ 95)?'

HOMEWORK

ATM simulator: ask for balance and withdrawal amount. Handle: sufficient funds, insufficient funds, negative withdrawal.

PCEP SYLLABUS COVERAGE

Section 3: Control flow — conditional statements

Trainer tip: Deliberately write bad indentation and show the `IndentationError`. Make it memorable — 'Python is strict about alignment.'

Day 6

Foundation

while Loops

LEARNING GOAL

Student understands loop mechanics, can write while loops, and avoids infinite loops.

TOPICS TO COVER

- while loop syntax and condition check
- Loop counter pattern
- break — exit the loop early
- continue — skip to next iteration
- Infinite loop risk and how to avoid it
- while True with break pattern

CODE EXAMPLE

```
# Count from 1 to 5
counter = 1
while counter <= 5:
    print(f"Count: {counter}")
    counter += 1

# Guess the number game
secret = 7
guess = 0
while guess != secret:
    guess = int(input("Guess: "))
    if guess < secret:
        print("Too low!")
    elif guess > secret:
        print("Too high!")
    print("Correct!")
```

IN-SESSION SCENARIO

Number guessing game — build it together step by step. They will naturally want to add hints, which leads them to discover `elif`.

HOMEWORK

Login system: keep asking for a password until the correct one ('python123') is entered. After 3 wrong attempts, print 'Account locked.'

PCEP SYLLABUS COVERAGE

Section 3: Control flow — loops

Trainer tip: Run an infinite loop once on purpose (`while True: print('hi')`). Show `Ctrl+C` to stop it. Turns a scary concept into a funny memory.

LEARNING GOAL

Student can use for loops with range(), iterate over strings, and know when to use for vs while.

TOPICS TO COVER

- for loop syntax
- range(stop), range(start, stop), range(start, stop, step)
- Iterating over a string character by character
- Nested for loops
- break and continue in for loops
- for vs while — when to use which

CODE EXAMPLE

```
# Basic range
for i in range(1, 6):
    print(i) # 1 2 3 4 5

# Step
for i in range(0, 20, 5):
    print(i) # 0 5 10 15

# Iterate string
word = "Python"
for letter in word:
    print(letter)

# Multiplication table
for i in range(1, 6):
    for j in range(1, 6):
        print(f"{i*j:3}", end="")
    print()
```

IN-SESSION SCENARIO

Times table generator — ask for a number (e.g. 7), print the full 1–10 table. Clean and practical.

HOMEWORK

FizzBuzz: print numbers 1–50. For multiples of 3 print 'Fizz', multiples of 5 print 'Buzz', both print 'FizzBuzz'.

PCEP SYLLABUS COVERAGE

Section 3: Control flow — for loops, range()

Trainer tip: FizzBuzz is a classic for a reason — it forces them to combine loops + conditionals. Do not skip it.

PHASE 2 — CORE PYTHON (DAYS 8–18)

LEARNING GOAL

Student can create, access, modify, and iterate over lists.

TOPICS TO COVER

- List creation with []
- Indexing (positive and negative)
- Slicing list[start:stop:step]
- Mutable — lists can be changed
- append(), insert(), remove(), pop()
- len(), sorted(), reversed()
- Iterating a list with for

CODE EXAMPLE

```
fruits = ["apple", "banana", "cherry"]

print(fruits[0]) # apple
print(fruits[-1]) # cherry
print(fruits[0:2]) # ['apple', 'banana']

fruits.append("mango")
fruits.insert(1, "blueberry")
fruits.remove("banana")
popped = fruits.pop()

print(len(fruits))
for fruit in fruits:
    print(fruit.upper())
```

IN-SESSION SCENARIO

Shopping cart: add items, remove an item, show the cart, show total count.

HOMEWORK

Given a list of 5 exam scores, print the highest, lowest, and average.

PCEP SYLLABUS COVERAGE

Section 4: Collections — lists

Trainer tip: Negative indexing confuses beginners. Draw it on screen: [a, b, c, d] then -4, -3, -2, -1 below each item.

Day 9

Core Python

Tuples & Strings (deep dive)

LEARNING GOAL

Student understands immutability, knows key string methods, and can unpack tuples.

TOPICS TO COVER

- Tuple creation with ()
- Immutable — cannot be changed
- Tuple unpacking
- When to use tuple vs list
- String methods: upper(), lower(), strip(), split(), replace(), find(), startswith(), endswith()
- String repetition and membership (in)

CODE EXAMPLE

```
# Tuple
coords = (10, 20)
x, y = coords # unpacking
print(f"x={x}, y={y}")

# String methods
sentence = " Hello, Python World! "
print(sentence.strip())
print(sentence.lower())
print(sentence.replace("Python", "Amazing"))
words = sentence.split(",")
print(words)
print("Python" in sentence) # True
```

IN-SESSION SCENARIO

Name formatter: take a messy name input (extra spaces, wrong case) and clean it up using string methods.

HOMEWORK

Take a sentence as input, count the words, reverse the words, and check if a given word exists in the sentence.

PCEP SYLLABUS COVERAGE

Section 4: Collections — tuples, strings

Trainer tip: Stress immutability with a live error: try to do `coords[0] = 99`. The `TypeError` makes the concept click instantly.

Day 10 Core Python Dictionaries

LEARNING GOAL

Student can create and work with dictionaries, understand key-value pairs, and iterate over them.

TOPICS TO COVER

- Dictionary creation with `{}`
- Accessing values with keys
- Adding and updating key-value pairs
- `keys()`, `values()`, `items()`
- `get()` for safe access
- Checking if a key exists with `in`
- Nested dictionaries

CODE EXAMPLE

```
student = {
    "name": "Alex",
    "age": 19,
    "gpa": 3.75,
    "courses": ["Math", "CS", "Physics"]
}

print(student["name"])
print(student.get("phone", "N/A")) # safe

student["email"] = "alex@uni.edu" # add
student["age"] = 20 # update

for key, value in student.items():
    print(f"{key}: {value}")
```

IN-SESSION SCENARIO

Student registry: store info for 3 students, look up a student by name, update a grade.

HOMEWORK

Word frequency counter: take a sentence, count how many times each word appears.

PCEP SYLLABUS COVERAGE

Section 4: Collections — dictionaries

Trainer tip: Dictionary vs list analogy: a list is a numbered locker, a dictionary is a named locker. You look things up by name, not position.

Day 11 Core Python Functions — Part 1

LEARNING GOAL

Student can define functions, use parameters, return values, and understand why functions matter.

TOPICS TO COVER

- def keyword and function structure
- Parameters vs arguments
- return statement
- Functions without return (returns None)
- Calling a function multiple times
- DRY principle — Don't Repeat Yourself
- Docstrings

CODE EXAMPLE

```
def greet(name, greeting="Hello"):
    """Greet a person with a custom
    message."""
    return f"{greeting}, {name}!"

def calculate_area(length, width):
    """Return area of a rectangle."""
    return length * width

print(greet("Alex"))
print(greet("Sam", "Good morning"))

area = calculate_area(5, 3)
print(f"Area: {area} sq units")
```

IN-SESSION SCENARIO

Build a calculator app: separate functions for add, subtract, multiply, divide. Then a main function that calls the right one.

HOMEWORK

Write a function that takes a list of numbers and returns the min, max, and average as a tuple.

PCEP SYLLABUS COVERAGE

Section 5: Functions — definition, parameters, return

Trainer tip: Show what happens when you forget return — the function returns None. This prevents a very common bug later.

Day 12

Core Python

Functions — Part 2

LEARNING GOAL

Student understands scope, *args, **kwargs, and can write more flexible functions.

TOPICS TO COVER

- Local vs global scope
- global keyword (use sparingly)
- *args — variable positional arguments
- **kwargs — variable keyword arguments
- Lambda functions for simple operations
- Functions calling other functions

CODE EXAMPLE

```
# Scope
x = 10 # global
def show():
    x = 20 # local
    print(x) # 20
show()
print(x) # 10

# *args
def total(*numbers):
    return sum(numbers)
print(total(1, 2, 3, 4, 5)) # 15

# Lambda
square = lambda n: n ** 2
print(square(7)) # 49

double = lambda x: x * 2
numbers = [1, 2, 3, 4]
print(list(map(double, numbers)))
```

IN-SESSION SCENARIO

Flexible discount calculator: takes a price plus any number of discount rates and applies them all.

HOMEWORK

Write a function using **kwargs that builds a profile dictionary from keyword arguments.

PCEP SYLLABUS COVERAGE

Section 5: Functions — scope, *args, **kwargs, lambda

Trainer tip: Scope is a top PCEP exam topic. Draw a box diagram: outer box = global, inner box = function. Variables stay in their box.

Day 13

Core Python

Error Handling

LEARNING GOAL

Student can anticipate errors, use try/except blocks, and write robust programs.

TOPICS TO COVER

- Types of errors: `SyntaxError`, `TypeError`, `ValueError`, `ZeroDivisionError`, `IndexError`, `KeyError`
- `try / except` block
- `except` with specific error types
- `else` clause in `try/except`
- `finally` clause — always runs
- Raising exceptions with `raise`

CODE EXAMPLE

```
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
        return None
    except TypeError:
        print("Error: Both inputs must be numbers!")
        return None
    else:
        print("Division successful.")
        return result
    finally:
        print("safe_divide() finished.")

print(safe_divide(10, 2))
print(safe_divide(10, 0))
```

IN-SESSION SCENARIO

Robust age input: keep asking until the user enters a valid integer > 0. Handle non-number input gracefully.

HOMEWORK

Wrap yesterday's calculator in `try/except` — handle division by zero and non-numeric input for all operations.

PCEP SYLLABUS COVERAGE

Section 6: Exceptions — `try/except`, error types

Trainer tip: PCEP loves asking which error is raised in specific situations. Build a cheat-sheet together: what causes each common error.

Day 14

Core Python

Review & Mini Project 1

LEARNING GOAL

Consolidate Days 1–13. Student builds a complete program using everything learned so far.

TOPICS TO COVER

- Revision: variables, types, input/output
- Revision: if/elif/else, while, for
- Revision: lists, dicts, functions, error handling
- Project: Student Grade Manager

CODE EXAMPLE

```
students = {}

def add_student(name, grades):
    students[name] = grades

def get_average(grades):
    return sum(grades) / len(grades)

def show_report():
    for name, grades in students.items():
        avg = get_average(grades)
        grade = "Pass" if avg >= 50 else "Fail"
        print(f"{name}: avg={avg:.1f} ({grade})")

add_student("Alex", [78, 85, 90])
add_student("Sam", [45, 52, 48])
show_report()
```

IN-SESSION SCENARIO

Let the student drive — you ask questions ('How would we add a student?') rather than typing. Coach, don't code for them.

HOMEWORK

Extend the grade manager: add a function to find the top student, and handle the case where a student has no grades.

PCEP SYLLABUS COVERAGE

All sections covered so far

Trainer tip: This day is diagnostic. Watch where they hesitate — that is tomorrow's revision target.

Day 15

Core Python

List Comprehensions

LEARNING GOAL

Student can write list comprehensions and understands when to use them vs regular loops.

TOPICS TO COVER

- Basic list comprehension syntax
- Comprehension with condition (filter)
- Nested list comprehension
- Dictionary comprehension
- Readability — when NOT to use comprehension
- map() and filter() briefly

CODE EXAMPLE

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Squares of all numbers
squares = [n**2 for n in numbers]

# Even numbers only
evens = [n for n in numbers if n % 2 == 0]

# Dict comprehension
word_len = {word: len(word)
            for word in ["apple", "kiwi", "cherry"]}

print(squares)
print(evens)
print(word_len)
```

IN-SESSION SCENARIO

Temperature converter: given a list of Celsius values, produce a list of Fahrenheit values in one line.

HOMEWORK

Given a list of words, use comprehension to get only words longer than 4 letters, in uppercase.

PCEP SYLLABUS COVERAGE

Section 4: Collections, comprehensions

Trainer tip: Show the equivalent for loop side by side with the comprehension. Let them appreciate the conciseness.

Day 16

Core Python

String Formatting & More Methods

LEARNING GOAL

Student masters all string formatting styles and advanced string methods relevant to PCEP.

TOPICS TO COVER

- f-strings (preferred)
- format() method
- % formatting (older, still on PCEP)
- Formatting numbers: .2f, :05d, :>10
- String multiplication
- join() and split()
- isdigit(), isalpha(), isspace()

CODE EXAMPLE

```
name = "Alex"
score = 95.678

# f-string
print(f"Name: {name}, Score:
{score:.2f}")

# format()
print("Name: {}, Score:
{:.2f}".format(name, score))

# % style
print("Name: %s, Score: %.2f" % (name,
score))

# Number formatting
print(f"{score:010.2f}") # 0000095.68

# join
words = ["Python", "is", "fun"]
print(" ".join(words))
```

IN-SESSION SCENARIO

Receipt printer: format a list of items with right-aligned prices and a total line.

HOMEWORK

Build a text-based table formatter: given a list of names and scores, print a neatly aligned table.

PCEP SYLLABUS COVERAGE

Section 2: Strings and formatting

Trainer tip: PCEP has tricky formatting questions. Practice reading format strings out loud until it becomes natural.

Day 17

Core Python

Modules & import

LEARNING GOAL

Student can import standard library modules, use them confidently, and understand module structure.

TOPICS TO COVER

- import module
- from module import name
- import module as alias
- math module: sqrt, ceil, floor, pi, pow
- random module: random(), randint(), choice(), shuffle()
- datetime module: basics
- Writing your own module (.py file)

CODE EXAMPLE

```
import math
from random import randint, choice
import datetime

# math
print(math.sqrt(144)) # 12.0
print(math.ceil(4.3)) # 5
print(math.pi) # 3.14159...

# random
print(randint(1, 100))
fruits = ["apple", "banana", "cherry"]
print(choice(fruits))

# datetime
today = datetime.date.today()
print(f"Today: {today}")
```

IN-SESSION SCENARIO

Dice rolling simulator: roll two dice, show the result, ask if they want to roll again.

HOMEWORK

Build a random quiz generator: store 5 questions in a list, randomly pick 3, ask them in order.

PCEP SYLLABUS COVERAGE

Section 5: Modules — import, standard library

Trainer tip: Show the student how to use `help(math)` in the shell. Teaching them to explore documentation is a skill that outlasts the exam.

Day 18

Core Python

File I/O

LEARNING GOAL

Student can read from and write to text files safely using context managers.

TOPICS TO COVER

- `open()` with modes: `r`, `w`, `a`, `r+`
- `with` statement — context manager (best practice)
- `read()`, `readline()`, `readlines()`
- `write()` and `writelines()`
- File not found handling
- Iterating over file lines
- Working with CSV files (basic)

CODE EXAMPLE

```
# Writing to a file
with open("notes.txt", "w") as f:
    f.write("Python is great\n")
    f.write("I am learning every day\n")

# Reading a file
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)

# Read line by line
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())
```

IN-SESSION SCENARIO

Student diary app: write entries to a file with today's date, read them back on demand.

HOMEWORK

Write a contact book that saves names and phone numbers to a file and loads them on startup.

PCEP SYLLABUS COVERAGE

Section 6: File operations

Trainer tip: Always teach with `open()` as `f` — never teach `open()` without it. The `with` pattern is the professional standard.

PHASE 3 — INTERMEDIATE & OOP BASICS (DAYS 19–25)

Day 19

Intermediate

Object-Oriented Programming — Part 1

LEARNING GOAL

Student understands classes, objects, and the `__init__` method.

TOPICS TO COVER

- What is OOP? Real world analogy
- class keyword
- __init__ constructor method
- self parameter
- Instance attributes vs class attributes
- Creating objects (instances)
- Calling methods on objects

CODE EXAMPLE

```
class Student:
    school = "City University" # class attribute

    def __init__(self, name, age, gpa):
        self.name = name
        self.age = age
        self.gpa = gpa

    def introduce(self):
        return f"Hi, I am {self.name}, GPA: {self.gpa}"

    def is_honours(self):
        return self.gpa >= 3.5

s1 = Student("Alex", 19, 3.8)
print(s1.introduce())
print(s1.is_honours()) # True
```

IN-SESSION SCENARIO

Model a BankAccount class: deposit, withdraw, check balance. Object = one account.

HOMEWORK

Create a Book class with title, author, year. Add a method that returns a formatted description.

PCEP SYLLABUS COVERAGE

Section 7: OOP basics — classes and objects

Trainer tip: Analogy: a class is a cookie cutter, an object is the cookie. Many cookies (objects) from one cutter (class).

Day 20

Intermediate

OOP — Part 2: Inheritance & Magic Methods

LEARNING GOAL

Student understands inheritance, can override methods, and knows key magic methods.

TOPICS TO COVER

- Inheritance — child class extends parent
- `super()` to call parent methods
- Method overriding
- `__str__` for print-friendly output
- `__len__`, `__add__` briefly
- `isinstance()` and `issubclass()`
- Why OOP matters

CODE EXAMPLE

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."
    def __str__(self):
        return f"Animal: {self.name}"

class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

animals = [Dog("Rex"), Cat("Whiskers")]
for animal in animals:
    print(animal.speak())
```

IN-SESSION SCENARIO

Vehicle hierarchy: Vehicle -> Car, Truck. Each has its own `describe()` method.

HOMEWORK

Create an `Employee` class and a `Manager` subclass. `Manager` has an extra method to add team members.

PCEP SYLLABUS COVERAGE

Section 7: OOP — inheritance, `__str__`

Trainer tip: PCEP does not go deep into OOP, but `__str__` and basic inheritance appear. Keep it practical.

Day 21

Intermediate

Sets

LEARNING GOAL

Student can create and use sets, understand uniqueness, and apply set operations.

TOPICS TO COVER

- Set creation with {} or set()
- Unordered and no duplicates
- add(), remove(), discard()
- Union |, Intersection &, Difference -
- Symmetric difference ^
- issubset(), issuperset()
- When to use a set vs list

CODE EXAMPLE

```
a = {1, 2, 3, 4, 5}
b = {3, 4, 5, 6, 7}

print(a | b) # union
print(a & b) # intersection
print(a - b) # difference
print(a ^ b) # symmetric diff

# Remove duplicates from list
names = ["Alex", "Sam", "Alex", "Jordan"]
unique = list(set(names))
print(unique)

print(3 in a) # True
```

IN-SESSION SCENARIO

Find common students enrolled in two different courses using set intersection.

HOMEWORK

Given two lists of numbers, find: numbers in both, numbers only in list 1, and all unique combined.

PCEP SYLLABUS COVERAGE

Section 4: Collections — sets

Trainer tip: Sets are $O(1)$ membership check vs $O(n)$ for lists. Mention this — even a beginner appreciates faster code.

Day 22

Intermediate

Recursion

LEARNING GOAL

Student understands what recursion is, can trace recursive calls, and write simple recursive functions.

TOPICS TO COVER

- What is recursion? A function calling itself
- Base case — stops the recursion
- Recursive case — moves toward base case
- Call stack mental model
- Factorial and Fibonacci as classics
- When recursion makes sense vs loops
- RecursionError and limits

CODE EXAMPLE

```
# Factorial
def factorial(n):
    if n == 0 or n == 1: # base case
        return 1
    return n * factorial(n - 1)

print(factorial(5)) # 120

# Countdown
def countdown(n):
    if n <= 0:
        print("Blast off!")
        return
    print(n)
    countdown(n - 1)

countdown(5)
```

IN-SESSION SCENARIO

Trace factorial(4) on screen together: 4x3x2x1. Draw each function call as a stack.

HOMEWORK

Write a recursive function to compute the nth Fibonacci number. Test for n=0 through n=10.

PCEP SYLLABUS COVERAGE

Section 5: Functions — recursion

Trainer tip: Draw the call stack visually on screen. Without visualisation, recursion is just confusing. With it, it is elegant.

Day 23

Intermediate

Iterators, Generators & enumerate/zip

LEARNING GOAL

Student understands Python's iteration protocol and can use enumerate, zip, and basic generators.

TOPICS TO COVER

- iter() and next()
- for loop is syntactic sugar for iter/next
- enumerate() — index + value
- zip() — pair two lists
- Generator functions with yield
- Generator expressions
- Lazy evaluation concept

CODE EXAMPLE

```
# enumerate
fruits = ["apple", "banana", "cherry"]
for i, fruit in enumerate(fruits,
start=1):
    print(f"{i}. {fruit}")

# zip
names = ["Alex", "Sam", "Jordan"]
scores = [88, 92, 75]
for name, score in zip(names, scores):
    print(f"{name}: {score}")

# Generator
def countdown(n):
    while n > 0:
        yield n
        n -= 1

for num in countdown(5):
    print(num)
```

IN-SESSION SCENARIO

Paired report card: zip student names with their grades and print a formatted result.

HOMEWORK

Write a generator that yields only even numbers from 1 to n. Collect first 5 values into a list.

PCEP SYLLABUS COVERAGE

Section 3/5: Iterators, generators

Trainer tip: enumerate() and zip() appear in real Python code constantly. Make sure these feel comfortable.

Day 24

Intermediate

Regular Expressions (intro)

LEARNING GOAL

Student understands what regex is and can use basic patterns for the PCEP exam.

TOPICS TO COVER

- What is regex? Pattern matching
- import re
- re.search(), re.match(), re.findall()
- Basic patterns: . * + ? ^ \$
- Character classes: \d \w \s
- Grouping with ()
- Practical use: validation

CODE EXAMPLE

```
import re

text = "My phone is 555-1234 and email
is hi@test.com"

# Find a phone pattern
phone = re.findall(r"\d{3}-\d{4}",
text)
print(phone) # ['555-1234']

# Email validation
def is_valid_email(email):
    pattern = r"[\w.-]+@[ \w.-]+\.\w+"
    return bool(re.match(pattern, email))

print(is_valid_email("hi@test.com")) #
True
print(is_valid_email("not-an-email")) #
False
```

IN-SESSION SCENARIO

Form validator: check that a username is 3-15 characters and contains only letters and numbers.

HOMEWORK

Write a function that extracts all numbers from a string using regex.

PCEP SYLLABUS COVERAGE

Section 6: String operations — regex basics

Trainer tip: PCEP regex is not deep — focus on \d, \w, \s, +, *, and re.search vs re.match.

Day 25

Intermediate

Review & Mini Project 2

LEARNING GOAL

Consolidate Days 14–24. Student builds a complete OOP-based application.

TOPICS TO COVER

- Revision: OOP, sets, recursion, modules
- Revision: file I/O, error handling
- Project: Library Management System

CODE EXAMPLE

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.is_borrowed = False
    def __str__(self):
        status = "borrowed" if self.is_borrowed
        else "available"
        return f"{self.title} by {self.author} [{status}]"

class Library:
    def __init__(self):
        self.books = []
    def add_book(self, book):
        self.books.append(book)
    def borrow(self, title):
        for book in self.books:
            if book.title == title:
                if not book.is_borrowed:
                    book.is_borrowed = True
                    return f"Borrowed: {title}"
                return "Already borrowed!"
        return "Book not found"
```

IN-SESSION SCENARIO

Build it together live. Student explains each decision — you ask 'why this approach?' to reinforce understanding.

HOMEWORK

Add a `return_book()` method and a `search_by_author()` method to the library system.

PCEP SYLLABUS COVERAGE

All sections

Trainer tip: If the student is struggling, scale down. Confidence matters more than elegance at this stage.

PHASE 4 — EXAM PREPARATION (DAYS 26–30)

Day 26

Exam Prep

PCEP Exam Topics Deep Dive

LEARNING GOAL

Identify and fill all remaining gaps specifically on PCEP syllabus topics.

TOPICS TO COVER

- PCEP exam structure: 30 questions, 45 minutes, 70% to pass
- Section weights: Fundamentals 18%, Control Flow 29%, Data Collections 25%, Functions 28%
- Common trap questions on operator precedence
- Type coercion traps
- Mutable default argument gotcha
- None vs False vs 0 vs empty string

CODE EXAMPLE

```
# PCEP trap: mutable default argument
def add_item(item, lst=[]): # BUG!
    lst.append(item)
    return lst

print(add_item("a")) # ['a']
print(add_item("b")) # ['a', 'b'] – surprise!

# Fix:
def add_item(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst

# Precedence trap
print(2 ** 3 ** 2) # 512, not 64! (right-assoc)
```

IN-SESSION SCENARIO

Do 10 real PCEP-style questions together. Student answers, you explain both right and wrong choices.

HOMEWORK

Student independently answers 15 PCEP practice questions from openedg.org.

PCEP SYLLABUS COVERAGE

All sections — exam focus

Trainer tip: Download the official PCEP exam syllabus from pythoninstitute.org. Go through it topic by topic and tick off what is covered.

Day 27

Exam Prep

Mock Exam 1 & Debrief

LEARNING GOAL

Full timed mock exam under real conditions. Identify weak areas for targeted revision.

TOPICS TO COVER

- 30 mock questions in 45 minutes
- Timed, closed notes
- Detailed debrief after
- Error pattern analysis
- Topic re-explanation as needed

CODE EXAMPLE

```
# Sample: output prediction
x = 5
y = 2
print(x // y, x % y, x ** y)
# Answer: 2 1 25

# Sample: error spotting
def greet(name):
    print("Hello", name) # IndentationError

# Sample: fill the blank
lst = [3, 1, 4, 1, 5, 9]
lst.____()
print(lst) # [1, 1, 3, 4, 5, 9]
# Answer: sort
```

IN-SESSION SCENARIO

Student takes the mock alone. You observe silently, note any long pauses. Debrief each answer with 'why is this right/wrong?'

HOMEWORK

Student revises the 3 weakest topics identified in today's debrief.

PCEP SYLLABUS COVERAGE

Full exam simulation

Trainer tip: Reproduce the exam environment: timer, no help, one screen. Anxiety about the format is real — practice it away.

Day 28

Exam Prep

Targeted Revision

LEARNING GOAL

Fix weak spots found in Mock 1. Deep practice on the lowest-scoring topics.

TOPICS TO COVER

- Personalised to student's Mock 1 results
- Re-teach any topic below 60% accuracy
- Fresh examples, different angles
- Common PCEP gotchas: is vs ==, mutable defaults, scope rules
- Short quizzes after each topic

CODE EXAMPLE

```
# is vs == (common PCEP trap)
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b) # True (same value)
print(a is b) # False (different object)
print(a is c) # True (same object!)

# String immutability
s = "hello"
s[0] = "H" # TypeError! Strings are immutable
```

IN-SESSION SCENARIO

Student teaches YOU the weak topics back. If they can explain it, they know it.

HOMEWORK

Rewrite all buggy or confusing code examples from Mock 1 from scratch, without looking at notes.

PCEP SYLLABUS COVERAGE

Personalised to gaps

Trainer tip: The reverse teaching technique is the most powerful revision method. A student who can explain it has truly understood it.

Day 29

Exam Prep

Mock Exam 2 & Speed Practice

LEARNING GOAL

Second full mock — target improvement from Mock 1. Focus on speed and confidence.

TOPICS TO COVER

- New 30-question mock, 45 minutes
- Strategy: answer easy ones first, flag hard ones
- Process of elimination technique
- Reading questions twice before answering
- Checking output-prediction questions step by step

CODE EXAMPLE

```
# Strategy: trace manually
i = 0
result = []
while i < 5:
    if i % 2 == 0:
        result.append(i)
    i += 1
print(result)

# Trace:
i=0(add),1(skip),2(add),3(skip),4(add)
# Answer: [0, 2, 4]

# 'What error' questions
lst = [1, 2, 3]
print(lst[5]) # IndexError, not
ValueError
```

IN-SESSION SCENARIO

After the mock, compare score with Mock 1. Celebrate improvement. Analyse remaining errors without judgment.

HOMEWORK

Review the PCEP syllabus one final time. Mark everything as confident / needs review / shaky.

PCEP SYLLABUS COVERAGE

Full exam simulation 2

Trainer tip: If the student scores above 75% on Mock 2, they are ready. If not, focus Day 30 on the shaky topics.

Day 30

Exam Prep

Final Revision & Exam Confidence

LEARNING GOAL

Last session: consolidate knowledge, build exam confidence, practical tips for exam day.

TOPICS TO COVER

- Quick-fire revision of all key topics
- Cheat sheet review (no new content today)
- Exam day logistics: arrive early, read carefully, manage time
- What to do when stuck: skip and return
- Positive mental preparation
- Next steps after PCEP

CODE EXAMPLE

```
# Quick-fire cheat sheet

# Type casting
int("5") -> 5 | str(5) -> "5" |
float("3.2") -> 3.2

# List methods
.append() .insert() .pop() .remove()
.sort() .reverse() .count() .index()

# String methods
.upper() .lower() .strip() .split()
.replace() .find() .startswith()
.join()

# Error types
# SyntaxError ValueError TypeError
# IndexError KeyError ZeroDivisionError
```

IN-SESSION SCENARIO

End with a celebration. Ask the student: 'What is the one thing you found hardest that you now fully understand?'

HOMEWORK

Get a good night's sleep. You are ready.

PCEP SYLLABUS COVERAGE

Full syllabus — final review

Trainer tip: Your job today is 30% content, 70% coach. The student knows the material. Your role is to convince them that they know it.